

# Telegram OSINT Collection Service

Telegram MTProto data-collection and access-management API — state of the implementation

Codebase telegram\_service v0.1.0    Document v1.0 · 6 Sep 2026    For Team Leader / Technical Lead    Basis full source, runtime database, test suite

Classification Internal — Confidential

35

REST ENDPOINTS

1

TELEGRAM ACCOUNT

12,805

MESSAGES COLLECTED

189

AUTOMATED TESTS

0

AUTH CONTROLS

## WHAT IT DOES

Turns **one authenticated Telegram user account** into a controlled, auditable data-collection API: registers channels, groups and bots as *sources*, determines each one's **real** access status by probing Telegram rather than assuming it, runs a legitimate join-request workflow for private sources, monitors accessible sources on a checkpointed 5-minute cycle, backfills history on demand, and searches Telegram live — storing results in SQLite with per-message raw evidence and SHA-256 hashes.

Scope is deliberately **collection and access management**; analysis (NLP, OCR, correlation, risk scoring) is downstream and not implemented here. **Stack**: FastAPI + Uvicorn, Telethon (MTProto, not the Bot API), SQLAlchemy 2 async over SQLite, APScheduler embedded in the API process.

## ARCHITECTURE & DATA FLOW

Client -> FastAPI router

-> Service layer

-> Telegram layer

| Repository layer

-> Telegram MTProto

validation + status codes only

business rules, access rules, audit

the ONLY Telethon importer

the ONLY query builder

SQLite + raw evidence JSON

Layering holds under inspection: routes never touch Telethon or SQL. Access is governed by a validated **13-state machine** that refuses to guess — a source becomes monitorable only after a real read probe succeeds.

## CAPABILITY POSITION — WHAT THE ACCOUNT CAN AND CANNOT REACH

| Capability                     | Status    | Determining factor                                                                                                                                                                          |
|--------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Read public channels / groups  | Supported | Verified read probe, then incremental collection with restart-safe checkpointing.                                                                                                           |
| Read private sources           | Partial   | Only after Telegram approves a join request. No bypass of any kind is attempted — by design.                                                                                                |
| Telegram-wide public search    | Partial   | <code>channels.searchPosts</code> is quota-gated by Telegram; the fallback <code>messages.searchGlobal</code> is <b>account-scoped</b> , not all of Telegram. Responses say which was used. |
| Join channels / invite links   | Supported | Exactly one join request per source, ever; no alternative route after a rejection.                                                                                                          |
| Detect a declined join request | Partial   | Invite links: yes. Username-based: <b>Telegram exposes no decline signal</b> — stays pending, accurately.                                                                                   |
| Send messages                  | Partial   | Exactly one literal <code>/start</code> to a registered bot. Reactions, polls, threads and member lists are not implemented.                                                                |

## FINDINGS REQUIRING A DECISION

| Severity | Finding                                                                                  | Impact and fix                                                                                                                                                                                                                                                                                                                                             |
|----------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High     | <b>No authentication on any endpoint</b> (SEC-01); default bind <code>0.0.0.0</code>     | Anyone reaching the port can make the account join channels, send <code>/start</code> , drive search until Telegram rate-limits it, read all collected third-party content, and delete sources with their data. Swagger UI at <code>/docs</code> is an unauthenticated console over it. <b>Fix</b> : API-key dependency, bind to loopback, TLS at a proxy. |
| High     | <b>Two endpoints return 200 without saving</b> (ISS-01)                                  | <code>PATCH /api/sources/{id}</code> and <code>POST /api/notifications/{id}/read</code> flush without committing, so the request-scoped session rolls back. Verified by reproduction. <b>Fix</b> : one line each, plus a regression test.                                                                                                                  |
| Medium   | No FloodWait enforcement or request budget                                               | Rate limits are reported but never enforced; only search concurrency (3) is capped — collection, joins and link extraction are uncapped. Account restriction is the realistic failure mode.                                                                                                                                                                |
| Medium   | Backfill does not survive a restart (ISS-02); documented fast path not wired in (DOC-01) | Backfill runs as an in-process task, sticking at <code>RUNNING</code> with no resume or cancel. The README's event-driven access detection exists as code but is never called, so approval latency is <b>up to 12 hours</b> .                                                                                                                              |
| Policy   | Data at rest, and user/DM sources are monitorable                                        | 12,805 messages plus raw evidence stored unencrypted with no retention policy. Separately, the generic source path can put a person or bot into monitoring, collecting one-to-one conversation history — very likely unintended.                                                                                                                           |

## STRENGTHS CONFIRMED IN CODE

- Access is never inferred** — a real probe drives a validated state machine; blocked transitions are refused, not forced.
- Restart-safe collection** — per-message commits plus a DB-level uniqueness constraint make re-collection a no-op.
- Evidence preservation** — raw payload stored and SHA-256 hashed per message; audit row on every significant operation.
- No bypass anywhere** — no invite guessing, no membership spoofing, no working around `ChannelPrivateError`, no automatic payment, no duplicate joins.
- Credential hygiene** — `api_hash`, session data, codes and 2FA passwords never logged or returned; phones masked; queries logged by length only.
- No SSRF or SQL-injection surface**; **189 tests** with a guard that fails the build if any published endpoint is untested.

## RECOMMENDED SEQUENCE

- Before any non-local deployment**
  - Add API authentication; bind to `127.0.0.1`; terminate TLS at a reverse proxy.
  - Fix the two missing commits (ISS-01) with regression tests; bound every `limit` parameter.
- Account safety**
  - Enforce FloodWait centrally instead of only reporting it.
  - Add a global outbound request budget covering collection, joins and link extraction; rate-limit joins; add jitter to collection cycles.
- Correctness and operations**
  - Backfill lifecycle: startup reconciliation, cancellation, per-source concurrency guard.
  - Alembic migrations and a dialect-agnostic upsert before any PostgreSQL move; wire in or delete the event handler and align the README with the code.

OVERALL ASSESSMENT

The architecture is **sound and unusually disciplined for a proof of concept**: strict layering that holds under inspection, a state machine that refuses to guess access, restart-safe checkpointing backed by a database-level uniqueness guarantee, evidence preservation, comprehensive auditing, and an explicit refusal to bypass Telegram's controls. **The gaps are operational, not architectural** — no authentication, two silent persistence bugs, and no enforcement of the rate-limit discipline the design otherwise respects. All are small, well-isolated changes. **Recommendation: keep the service on a trusted network only; fix ISS-01 and add authentication before any wider deployment; then address the account-safety backlog before increasing collection volume.**